
LABORATORY ASSIGNMENT

on

Object Oriented Programming (PCC-CS593)

Computer Science & Engineering

5TH Semester



Department of Computer Science & Engineering

Academy of Technology

Hooghly – 712121

West Bengal, India

Course Outcomes:

After completion of this course, the learners will be able to

CO	Course Outcome	BTL
CO1	Explain the fundamental concepts of Java programming and object-oriented programming, including classes, objects, methods, constructors, encapsulation, and polymorphism.	L2 – Understand
CO2	Implement object-oriented Java applications using classes, objects, constructors, inheritance, interfaces, abstraction, polymorphism, and packages.	L3 – Apply
CO3	Analyze Java applications using strings, generics, collections, iterators, exception handling, and other object-oriented programming features to solve programming problems.	L4 – Analyze
CO4	Evaluate Java applications involving multithreading and collection-based programming by analyzing their execution, performance, and suitability for given problems.	L5 – Evaluate
CO5	Design and develop GUI-based Java applications using Java Swing and event-driven programming principles to solve practical problems.	L6 – Create

CO–PO Mapping Matrix

Correlation Levels:

- 3 = High
- 2 = Moderate
- 1 = Low
- – = No Significant Correlation

CO / PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO1	3	2	2	1	3	–	–	1	1	–	2
CO2	3	3	3	1	3	–	1	2	1	1	2
CO3	3	3	2	2	3	1	1	1	1	–	2
CO4	3	3	3	3	3	1	1	2	2	1	3
CO5	3	3	3	2	3	2	1	3	3	2	3

CO–PSO Mapping

Assignment 1:

Title: Java Basics, Object-Oriented Programming Concepts, Classes and Objects

Duration: 1 Laboratory Sessions (2 Hours)

Total Marks: 30

CO Mapping: CO1

Bloom's Taxonomy Levels Covered: L3, L4, L5

Assessment Pattern: Programming – 20 | Analysis – 5 | Evaluate – 5

Part A – Programming Exercises - 20 Marks | BTL: L3 – Apply

Students should write, compile, execute, and demonstrate all five programs.

Program 1 – Basic Java Program - 4 Marks

Define a class Calculation and include two methods to calculate

- a) factorial of an integer number
- b) GCD of two integer numbers

Program 2 – Class and Object - 4 Marks

Write a class program with the following specifications:

Class Name — Triplet

Data Members — int a, b, c

Member Methods:

void getdata() — to accept three numbers

void findprint() — to check and display whether the numbers are Pythagorean Triplets or not.

Program 3 – Class and Objects - 4 Marks

Write a class program to accept two numbers as instance variables. Use the following functions for the given purposes:

Class Name — Calculate

void inputdata() — to input both the values

void calculate() — to find sum and difference

void outputdata() — to print sum and difference of both the numbers

Use a main method to call the functions.

Program 4 – Methods and Object Interaction - 4 Marks

Create a class Rectangle with:

- length
- breadth

Implement the following methods:

readData()

calculateArea()

calculatePerimeter()

display()

Create two objects and display their area and perimeter.

Program 5 - Define a class Employee having the following description - 4 Marks

Data Members	Purpose
int pan	To store personal account number
String name	To store name
double taxincome	To store annual taxable income

double tax	To store tax that is calculated
Member functions	Purpose
void input()	Store the pan number, name, taxable income
void cal()	Calculate tax on taxable income
void display()	Output details of an employee

Calculate tax based on the given conditions and display the output as per the given format.

Total Annual Taxable Income	Tax Rate
Up to ₹2,50,000	No tax
From ₹2,50,001 to ₹5,00,000	10% of the income exceeding ₹2,50,000
From ₹5,00,001 to ₹10,00,000	₹30,000 + 20% of the income exceeding ₹5,00,000
Above ₹10,00,000	₹50,000 + 30% of the income exceeding ₹10,00,000

Output:

Pan Number Name Tax-Income Tax

.....

.....

Part B – Program Analysis and Debugging - 5 Marks | BTL: L4 – Analyze

Question 1 – Output Analysis - 2 Marks

Predict the output and justify your answer.

```
class Test {
    int x = 10;
    public static void main(String[] args) {
        Test t1 = new Test();
        Test t2 = t1;
        t2.x = 25;
        System.out.println(t1.x);
        System.out.println(t2.x);
    }
}
```

Students must explain:

- How many objects are created?
- How many reference variables are created?
- What object does t1 refer to?
- What object does t2 refer to?
- Why are both displayed values the same?

Question 2 – Debugging - 3 Marks

The following program contains errors:

```
class Student {
    int roll;
    String name;
    display() {
        System.out.println(roll);
        System.out.println(name);
    }
    public static void main(String[] args) {
        Student s = new Student();
        s.display();
    }
}
```

Tasks

- a) Identify the error(s).
- b) Explain why the error occurs.
- c) Rewrite the corrected program.
- d) Execute the corrected program and verify the output.

Part D – Evaluation Question - BTL: L5 – Evaluate | 5 Marks

Question – Evaluate the use of Class and Object Design

Consider the following two approaches for calculating the area and perimeter of rectangles.

Approach A: Use separate variables and calculations directly inside the main() method.

Approach B: Create a Rectangle class containing length, breadth, calculateArea(), calculatePerimeter(), and display() methods, and create objects for different rectangles.

Tasks:

- a) Compare Approach A and Approach B with respect to:
 - Reusability
 - Modularity
 - Maintainability
- b) Decide which approach is more suitable for developing a larger application.
- c) Justify your choice with at least two technical reasons.

Part E – Viva Voce - BTL: L1–L2 (Will be assessed during PCA1/PCA2)

L1 – Remember

1. What is Java?
2. What is a class?
3. What is an object?
4. Which keyword is used to create an object?
5. What is JVM?
6. What is the extension of a Java source file?
7. What is the entry point of a Java application?
8. What is a method?
9. What is an instance variable?
10. What is an object reference?

L2 – Understand

11. Differentiate between a class and an object.
12. Why is Java called platform independent?
13. Explain the purpose of the new keyword.
14. What is an object reference?
15. Explain the difference between an instance variable and a local variable.
16. Can two reference variables refer to the same object? Explain.
17. Why are methods used inside a class?
18. Explain the relationship between a class, object, and method.
19. Why are data members generally declared inside a class?
20. What is the difference between an object and an object reference?
21. What happens when two references refer to the same object?
22. Why is the main() method declared as static?
23. What is the purpose of the public keyword in the main() method?
24. What is the role of the JVM in executing a Java program?
25. Why is a class considered a blueprint for objects?

Assignment 2:

Title: Constructors, Constructor Overloading, this, this(), Method Overloading, Passing and Returning Objects

Duration: 1 Laboratory Session (2 Hours)

Total Marks: 30

CO Mapping: CO1

BTL Levels Assessed: L3, L4, L5

Part A – Programming Exercises - 20 Marks | BTL: L3 – Apply

Students should write, compile, execute, test and demonstrate all five programs.

Program 1 – Constructor and this Keyword - 4 Marks

Define a class Student with:

```
int rollNo
String name
String department
```

Implement:

- A default constructor.
- A parameterized constructor.
- Use the `this` keyword to distinguish instance variables from constructor parameters.
- A `display()` method.

Create objects using both constructors.

Example

```
Student s1 = new Student();
Student s2 = new Student(101, "Rahul", "CSE");
```

Demonstrate

- Default constructor
- Parameterized constructor
- `this`
- Object initialization

Program 2 – Constructor Overloading and this () - 4 Marks

Create a class Box with:

```
double length
double breadth
double height
```

Define the following overloaded constructors:

```
Box()
Box(double side)
Box(double length, double breadth, double height)
Box(Box b)
```

Use constructor chaining with `this()` wherever appropriate.

Implement:

```
double volume()
void display()
```

Create objects using all three constructors.

Example

```
Box b1 = new Box();
Box b2 = new Box(5);
```

```
Box b3 = new Box(5, 4, 3);
Box b4 = new Box(b3);
```

You must explain

- a) What is constructor overloading?
- b) Why is `this()` used?
- c) Difference between `this` and `this()`.
- d) How constructor chaining takes place.

Important distinction: `this` refers to the current object, whereas `this()` invokes another constructor of the same class.

Program 3 – Method Overloading - 4 Marks

Create a class `Calculator` and implement overloaded methods named `add()`:

```
int add(int a, int b)
double add(double a, double b)
int add(int a, int b, int c)
```

Invoke all three methods from `main()` and display their results.

Example

```
Calculator c = new Calculator();
System.out.println(c.add(10, 20));
System.out.println(c.add(10.5, 20.5));
System.out.println(c.add(10, 20, 30));
```

Students should demonstrate

- a) Same method name
- b) Different parameter list
- c) Compile-time method selection
- d) Method overloading

Concepts Assessed: Method overloading → Compile-time polymorphism

Program 4 – Passing and Returning Objects: Complex Numbers - 4 Marks

Create a class `Complex` with:

```
double real
double imaginary
```

Implement:

```
Complex()
Complex(double, double)
void showComplex()
Complex addComplex(Complex c)
```

The method: `Complex addComplex(Complex c)`
must:

- a) Accept another `Complex` object.
- b) Add the corresponding real and imaginary parts.
- c) Create a new `Complex` object containing the result.
- d) Return the resulting object.

Example

```
Complex 1 = 3 + 4i
Complex 2 = 5 + 6i
```

Output:

```
Sum = 8 + 10i
```

Students must identify

- a) Which object is passed to the method.
- b) Which object is represented by `this`.
- c) Which object is created inside the method.
- d) Which object is returned.
- e) How the returned object is received in `main()`.

Example:

```
Complex c3 = c1.addComplex(c2);
```

Program 5 – Method Overloading with Objects: Distance - 4 Marks

Create a class `Distance` containing:

```
int feet
int inches
```

Implement overloaded methods to demonstrate **method overloading with objects**.

Method 1: `Distance add(Distance d)`

This method accepts another `Distance` object and returns the sum.

Method 2: `Distance add(int feet, int inches)`

This method accepts distance values directly and returns the resulting `Distance` object.

Requirements: Perform proper conversion:

12 inches = 1 foot

Example:

Distance 1 = 5 feet 8 inches

Distance 2 = 7 feet 9 inches

Output:

Total Distance = 13 feet 5 inches

Also demonstrate:

```
Distance d3 = d1.add(d2);
```

```
Distance d4 = d1.add(7, 9);
```

Concepts Assessed:

- Method overloading
- Passing objects
- Passing primitive values
- Returning objects
- Object interaction
- Compile-time polymorphism

Part B – Program Analysis and Debugging - 5 Marks | BTL: L4 – Analyze

Question 1 – Analyze Constructor Chaining - 2 Marks

Predict the output and explain the order in which the constructors are executed.

```
class Test {
    int x;
    int y;
    Test() {
        this(10);
        System.out.println("Default");
    }
    Test(int x) {
        this(x, 20);
        System.out.println("One Argument");
    }
    Test(int x, int y) {
```

```

        this.x = x;
        this.y = y;
        System.out.println("Two Arguments");
    }
    void display() {
        System.out.println(x + " " + y);
    }
    public static void main(String[] args) {
        Test t = new Test();
        t.display();
    }
}

```

You must explain:

- Which constructor is called initially?
- How does `this()` transfer control?
- What is the order of constructor execution?
- What are the final values of `x` and `y`?

Question 2 – Debug Method Overloading - 3 Marks

The following program contains an error:

```

class Calculator {
    int add(int a, int b) {
        return a + b;
    }
    double add(int a, int b) {
        return a + b;
    }
    public static void main(String[] args) {
        Calculator c = new Calculator();
        System.out.println(c.add(10, 20));
    }
}

```

Tasks

- Identify the error.
- Explain why the methods cannot be overloaded in this way.
- Correct the program by modifying the parameter list.
- Compile and execute the corrected program.
- Explain why the corrected methods are considered overloaded.

Key concept

Students should understand: Changing only the return type does not constitute method overloading.

Part C – Evaluation - 5 Marks | BTL: L5 – Evaluate

Question – Evaluate Different Methods of Object Creation and Calculation

Consider the following three approaches for adding two complex numbers.

Approach A: Perform all calculations directly inside `main()` using primitive variables.

Approach B: Create a `Complex` class and perform addition using:

```
Complex c3 = c1.addComplex(c2);
```

Approach C: Create a `Complex` class with multiple overloaded `addComplex()` methods:

```
addComplex(Complex c)
```

```
addComplex(double real, double imaginary)
```

Tasks:

Evaluate the three approaches with respect to:

- a) Encapsulation
- b) Reusability
- c) Readability
- d) Maintainability
- e) Object-oriented design

Then answer:

Which approach would you recommend for developing a larger object-oriented application? Justify your answer with at least two technical reasons.

Part D – Viva Voce: BTL: L1–L2 (Will be assessed during PCA1/PCA2)

The following questions can be asked during the laboratory demonstration.

L1 – Remember

- 1. What is a constructor?
- 2. What is constructor overloading?
- 3. What is method overloading?
- 4. What is the `this` keyword?
- 5. What is `this()`?
- 6. What is constructor chaining?
- 7. Can a method return an object?
- 8. Can an object be passed as a method argument?
- 9. What is compile-time polymorphism?
- 10. What is an object reference?

L2 – Understand

- 11. Differentiate between `this` and `this()`.
- 12. Why is `this()` required to be the first statement of a constructor?
- 13. What is the difference between a constructor and a method?
- 14. Can constructors be overloaded? Explain.
- 15. Can a constructor be inherited?
- 16. Why is method overloading called compile-time polymorphism?
- 17. Can two methods be overloaded by changing only their return types?
- 18. How is an object passed to a method?
- 19. How does a method return an object?
- 20. What is the role of `this` in `Complex addComplex(Complex c)?`
- 21. Explain: `Complex c3 = c1.addComplex(c2);`
- 22. What is the difference between passing a primitive value and passing an object?
- 23. Why would a method return an object?
- 24. What is the difference between constructor overloading and method overloading?
- 25. Explain constructor chaining with an example.

Assignment 3:

Title: Static Members, Access Modifiers, Encapsulation, Getters/Setters, Aggregation, Composition and Object Relationships

Duration: 1 Laboratory Session (2 Hours)

Total Marks: 30

CO Mapping: CO2

BTL Levels Assessed: L3, L4, L5

Part A – Programming Exercises - 20 Marks | BTL: L3 – Apply

Students should write, compile, execute, test and demonstrate all five programs.

Program 1 – Static Data Member and Static Method - 4 Marks

Create a class Student with the following members:

```
private int rollNo;  
private String name;  
private static String collegeName;  
private static int studentCount;
```

Implement:

```
void input()  
void display()  
static void setCollegeName(String name)  
static void displayStudentCount()
```

Requirements

- Create at least three Student objects.
- Assign the same college name to all students using the static method.
- Increment studentCount whenever a Student object is created.
- Display individual student information.
- Display the total number of Student objects created.

Students should demonstrate

- Static data member
- Static method
- Instance data member
- Difference between class-level and object-level data

Program 2 – Encapsulation Using Access Modifiers and Getters/Setters - 4 Marks

Create a class BankAccount with the following private data members:

```
private int accountNo;  
private String accountHolder;  
private double balance;
```

Implement appropriate getter and setter methods:

```
setAccountNo()  
getAccountNo()
```

```
setAccountHolder()  
getAccountHolder()
```

```
setBalance()
```

```
getBalance()
```

Also implement:

```
deposit(double amount)
```

```
withdraw(double amount)
```

```
display()
```

Requirements

- Data members must be declared private.
- Access to the data members must be provided through methods.
- deposit() should reject zero or negative amounts.
- withdraw() should not allow withdrawal greater than the available balance.
- Demonstrate the use of getter and setter methods.

Program 3 – Uses-a Relationship: Bill and Product - 4 Marks

Create two classes:

Class Product

Data members:

```
private int productId;
```

```
private String productName;
```

```
private double price;
```

Provide an appropriate parameterized constructor and methods:

```
getProductId()
```

```
getProductName()
```

```
getPrice()
```

```
displayProduct()
```

Class Bill

Implement a method:

```
double calculateAmount(Product p, int quantity)
```

The method should:

- Accept a Product object as a parameter.
- Accept the quantity purchased.
- Calculate the total amount using:

Total Amount = Product Price × Quantity

- Return the calculated amount.

Also implement:

```
displayBill(Product p, int quantity)
```

 to display the product details, quantity and total amount.

Requirements

Students should:

- Create at least two Product objects independently.
- Create a Bill object.
- Pass a Product object to the Bill method.
- Calculate and display the bill amount.
- Demonstrate that the Bill class uses the Product object but does not permanently contain it.
- Explain why this relationship is called a Uses-a / Dependency relationship.

Program 4 – Aggregation: Department and Students - 4 Marks

Create the following classes:

```
Student
```

```
Department
```

Student Data members:

```
private int rollNo;  
private String name;
```

Provide appropriate constructors and methods.

Department Data members:

```
private String departmentName;  
private Student[] students;
```

Implement methods:

```
addStudent()  
displayStudents()
```

A Department object should contain multiple Student objects.

Requirements

- a) Create several Student objects.
- b) Create one Department object.
- c) Add the Student objects to the Department.
- d) Display all students belonging to the Department.
- e) Explain why the relationship represents aggregation.

Program 5 – Composition: Car and Engine - 4 Marks

Create two classes:

Engine

Car

Engine Implement:

```
private String engineNo;  
private int capacity;  
with appropriate constructor and methods.
```

Car Implement:

```
private String registrationNo;  
private Engine engine;
```

The Engine object should be created as part of the Car object.

Implement:

```
displayCarDetails()
```

Requirements

- a) Create an Engine as part of a Car.
- b) Associate the Engine with the Car.
- c) Display both Car and Engine details.
- d) Explain the relationship between Car and Engine.
- e) Explain why this relationship can be considered composition.

Part B – Program Analysis and Debugging - 5 Marks | BTL: L4 – Analyze

Question 1 – Static vs Instance Members - 2 Marks

Analyze the following program:

```
class Student {  
    int rollNo;  
    static String college = "AOT";  
    Student(int rollNo) {  
        this.rollNo = rollNo;  
    }  
    void display() {  
        System.out.println(rollNo + " " + college);  
    }  
}
```

```

    }
    public static void main(String[] args) {
        Student s1 = new Student(101);
        Student s2 = new Student(102);
        s1.college = "ABC College";
        s1.display();
        s2.display();
    }
}

```

Tasks

- Predict the output.
- Explain why both objects display the same college name.
- Identify which variable belongs to the object.
- Identify which variable belongs to the class.
- Explain what happens when s1.college is modified.

Question 2 – Analyze Aggregation and Composition - 3 Marks

Consider the following two designs:

Design A

Department ——— Student

Students can exist independently of the Department.

Design B

Car ♦—— Engine

The Engine is treated as a component owned by the Car.

Tasks

- Identify which relationship represents aggregation.
- Identify which relationship represents composition.
- Justify your answer based on object dependency/lifecycle.
- Give one real-world example of aggregation.
- Give one real-world example of composition.

Part C – Evaluation - 5 Marks | BTL: L5 – Evaluate

Question – Evaluate Encapsulation and Direct Data Access

Consider the following two designs.

Approach A – Public Data Members

```

class Student {
    public int rollNo;
    public String name;
    public double marks;
}

```

Approach B – Private Data with Getters/Setters

```

class Student {
    private int rollNo;
    private String name;
    private double marks;
    public int getRollNo() {
        return rollNo;
    }
    public void setRollNo(int rollNo) {
        this.rollNo = rollNo;
    }
}

```

```
    }  
    // Other getters/setters  
}
```

Tasks

Evaluate both approaches with respect to:

- a) Data hiding
- b) Security/control
- c) Maintainability
- d) Validation
- e) Flexibility of implementation

Then answer:

Which approach is more appropriate for a well-designed object-oriented application? Justify your answer with at least two technical reasons.

Part D – Viva Voce - BTL: L1–L2 (Will be assessed during PCA1/PCA2)

The following questions may be asked during the laboratory demonstration.

L1 – Remember

1. What is encapsulation?
2. What is data hiding?
3. What is an access modifier?
4. Name the access modifiers in Java.
5. What is a static variable?
6. What is a static method?
7. What is a getter?
8. What is a setter?
9. What is aggregation?
10. What is composition?

L2 – Understand

11. What is the difference between static and instance variables?
12. Why are static members shared among objects?
13. Can a static method directly access an instance variable?
14. Why are data members generally declared private?
15. What is the purpose of getter and setter methods?
16. How does a setter provide controlled access to an object's data?
17. Differentiate between aggregation and composition.
18. What is a has-a relationship?
19. Give an example of aggregation.
20. Give an example of composition.
21. Why is Department–Student generally modeled as aggregation?
22. Why can Car–Engine be modeled as composition?
23. What is the difference between private and public?
24. What is the purpose of protected?
25. What happens to a static variable when multiple objects are created?

Assignment 4:

Title: Inheritance, super Keyword, Constructor Inheritance, Method Overriding and Runtime Polymorphism

Duration: 1 Laboratory Session (2 Hours)

Total Marks: 30

CO Mapping: CO3

BTL Levels Covered: L3, L4, L5

Part A – Programming Exercises - 20 Marks | BTL: L3 – Apply

Students should write, compile, execute, test and demonstrate all five programs.

Program 1 – Single Inheritance - 4 Marks

Create two classes:

Person

Student

Data members of Class Person:

```
protected String name;
```

```
protected int age;
```

Methods:

```
void setPersonData()
```

```
void displayPersonData()
```

Data members of Class Student:

```
private int rollNo;
```

```
private String course;
```

Methods:

```
void setStudentData()
```

```
void displayStudentData()
```

Use:

```
class Student extends Person
```

Requirements

- Create a Student object.
- Initialize both Person and Student details.
- Display all details.
- Demonstrate that Student inherits members from Person.

Program 2 – super Keyword and Constructor Chaining - 4 Marks

Create two classes:

Employee

Manager

Class Employee

Data members:

```
protected int empId;
```

```
protected String name;
```

```
protected double basicSalary;
```

Create a parameterized constructor:

```
Employee(int empId, String name, double basicSalary)
```

and method:

```
void displayEmployee()
```

Class Manager

Additional data member:

```
private double bonus;
```

Create a constructor:

```
Manager(int empId, String name, double basicSalary, double bonus)
```

Use: `super(empId, name, basicSalary);` to invoke the superclass constructor.

Implement: `void displayManager()` to display complete information.

Requirements:

- a) Students must demonstrate:
- b) extends
- c) `super()`
- d) Constructor chaining
- e) Accessing superclass members
- f) Parent and child class constructors

Program 3 – Method Overriding - 4 Marks

Create the following classes:

Shape

Circle

Rectangle

Class Shape

Define: `void calculateArea()` with a general message.

Class Circle

Extend Shape and override: `void calculateArea()` to calculate the area of a circle.

Class Rectangle

Extend Shape and override: `void calculateArea()` to calculate the area of a rectangle.

Requirements

Create objects of:

Circle

Rectangle

and invoke `calculateArea()`.

Students should demonstrate that the subclass provides its own implementation of the superclass method.

Program 4 – Multilevel Inheritance - 4 Marks

Create the following inheritance hierarchy:

Person

↓

Employee

↓

Manager

Person

```
protected String name;  
protected int age;
```

Method:

```
void displayPerson()
```

Employee

Additional members:

```
protected int empId;  
protected double salary;
```

Method:

```
void displayEmployee()
```

Manager

Additional member:

```
private String department;
```

Method:

```
void displayManager()
```

Create a Manager object and display all information.

Requirements

Students should demonstrate that:

Manager → Employee → Person

and the Manager object can access inherited behavior from both levels.

Program 5 – Runtime Polymorphism / Dynamic Method Dispatch - 4 Marks

Create a superclass Employee with method:

```
void calculateSalary()
```

Create two subclasses: PermanentEmployee and ContractEmployee

Override calculateSalary() in both subclasses.

For example:

Permanent Employee: Salary = Basic Salary + Allowance

Contract Employee: Salary = Working Days × Daily Rate

In main(), create:

```
Employee e;
```

Then assign:

```
e = new PermanentEmployee(...);
```

```
e.calculateSalary();
```

```
e = new ContractEmployee(...);
```

```
e.calculateSalary();
```

Requirements

- Students must explain:
- Why the reference variable is of type Employee.

- c) Why different subclass methods are executed.
- d) What is runtime polymorphism?
- e) What is dynamic method dispatch?

Part B – Program Analysis and Debugging 5 Marks | BTL: L4 – Analyze

Question 1 – Analyze Constructor Execution - 2 Marks

Predict the output and explain the order of constructor execution.

```
class A {  
    A() {  
        System.out.println("Constructor A");  
    }  
}  
class B extends A {  
    B() {  
        System.out.println("Constructor B");  
    }  
}  
class C extends B {  
    C() {  
        System.out.println("Constructor C");  
    }  
}  
public class Test {  
    public static void main(String[] args) {  
        C obj = new C();  
    }  
}
```

Students must explain:

- a) Which constructor executes first?
- b) Why does A() execute before B()?
- c) Why does B() execute before C()?
- d) What is the role of super() in this process?

Question 2 – Analyze Method Overriding and Runtime Polymorphism - 3 Marks

Consider:

```
class Animal {  
    void sound() {  
        System.out.println("Animal sound");  
    }  
}  
class Dog extends Animal {  
    void sound() {  
        System.out.println("Dog barks");  
    }  
}  
class Cat extends Animal {  
    void sound() {  
        System.out.println("Cat meows");  
    }  
}
```

```

public class Test {
    public static void main(String[] args) {
        Animal a;
        a = new Dog();
        a.sound();
        a = new Cat();
        a.sound();
    }
}

```

Tasks

- Predict the output.
- Identify the reference type.
- Identify the actual object in each case.
- Explain why different versions of sound() are executed.
- Identify the OOP principle demonstrated.

Part C – Evaluation - 5 Marks | BTL: L5 – Evaluate

Question – Overloading vs Overriding: Consider the following two approaches.

Approach A – Method Overloading

```

class Calculator {
    int add(int a, int b) {
        return a + b;
    }
    double add(double a, double b) {
        return a + b;
    }
}

```

Approach B – Method Overriding

```

class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}
class Dog extends Animal {
    void sound() {
        System.out.println("Dog barks");
    }
}

```

Tasks

Evaluate the two approaches with respect to:

- When the method is selected
- Inheritance requirement
- Polymorphism
- Flexibility
- Runtime behavior

Then answer:

For a software system in which different subclasses must provide their own implementation of the same operation, would you recommend method overloading or method overriding? Justify your answer.

Part D – Viva Voce | BTL: L1–L2 (Will be assessed during PCA1/PCA2)

The following questions may be asked during laboratory demonstration.

L1 – Remember

- 1) What is inheritance?
- 2) What is a superclass?
- 3) What is a subclass?
- 4) Which keyword is used for inheritance in Java?
- 5) What is the super keyword?
- 6) What is method overriding?
- 7) What is runtime polymorphism?
- 8) What is dynamic method dispatch?
- 9) What is an is-a relationship?
- 10) Name the types of inheritance supported through classes in Java.

L2 – Understand

- 1) Why is inheritance used?
- 2) What is the difference between a superclass and subclass?
- 3) Explain the extends keyword.
- 4) What is the purpose of super()?
- 5) Why must super() be the first statement in a constructor?
- 6) What happens when a child class object is created?
- 7) Differentiate between method overloading and overriding.
- 8) Can a static method be overridden?
- 9) What is runtime polymorphism?
- 10) Explain dynamic method dispatch with an example.
- 11) What is upcasting?
- 12) Why is Employee e = new Manager(); possible?
- 13) What is the advantage of using a superclass reference?
- 14) What is the difference between aggregation and inheritance?
- 15) Explain the difference between has-a and is-a relationships.

Assignment 5:

Title: Abstract Classes, Abstract Methods, final Keyword and Polymorphic Design

Duration: 1 Laboratory Session (2 Hours)

Total Marks: 30

CO Mapping: CO3

BTL Levels Covered: L3, L4, L5

Part A – Programming Exercises - 20 Marks | BTL: L3 – Apply

Students should write, compile, execute, test and demonstrate all five programs.

Program 1 – Abstract Class and Abstract Method - 4 Marks

Create an abstract class Shape containing:

```
abstract class Shape
```

Data member:

```
protected String color;
```

Methods:

```
void setColor(String color)
```

```
abstract void calculateArea()
```

```
void displayColor()
```

Create two subclasses:

```
Circle
```

```
Rectangle
```

Circle Data member:

```
private double radius;
```

Override:

```
calculateArea()
```

Rectangle Data members:

```
private double length;
```

```
private double breadth;
```

Override:

```
calculateArea()
```

Requirements

- Create objects of Circle and Rectangle.
- Set their colors.
- Calculate and display their respective areas.
- Demonstrate implementation of an abstract method.
- Explain why an object of Shape cannot be created directly.

Program 2 – Abstract Class with Concrete Methods - 4 Marks

Create an abstract class Employee containing:

```
protected int empId;
```

```
protected String name;
```

Methods:

```
void displayEmployee()
```

```
abstract double calculateSalary()
```

Create two subclasses:

```
PermanentEmployee
```

```
ContractEmployee
```

PermanentEmployee Data members:

```
private double basicSalary;
```

```
private double allowance;
```

Calculate salary as:

Salary = Basic Salary + Allowance

ContractEmployee Data members:

```
private int workingDays;
```

```
private double dailyRate;
```

Calculate salary as:

Salary = Working Days × Daily Rate

Requirements

- Create objects of both subclasses.
- Display employee details.
- Calculate salary using the overridden `calculateSalary()`.
- Identify which method is abstract.
- Identify which method is concrete.

Program 3 – Runtime Polymorphism Using Abstract Class - 4 Marks

Create an abstract class:

```
abstract class Payment
```

with:

```
abstract void makePayment(double amount);
```

Create the following subclasses:

```
CreditCardPayment
```

```
UPIPayment
```

Override `makePayment()` in both classes.

In `main()` declare:

```
Payment p;
```

Then execute:

```
p = new CreditCardPayment();
```

```
p.makePayment(5000);
```

```
p = new UPIPayment();
```

```
p.makePayment(2500);
```

Requirements

Students must demonstrate:

- Abstract class reference.
- Different subclass objects.
- Method overriding.
- Runtime polymorphism.
- Dynamic method dispatch.

Program 4 – final Variable, Method and Class - 4 Marks

Create a class `University` containing:

```
final String UNIVERSITY_NAME = "MAKAUT";
```

Create a method: `final void displayRules()`

Create a subclass: `College`

Requirements

- Access the final variable.
- Display its value.
- Call the final method from the subclass.

- d) Attempt to modify the final variable and observe the compilation error.
- e) Attempt to override the final method and observe the compilation error.
- f) Explain why the compiler prevents these operations.

Then create another class:

```
final class SecuritySystem
```

Attempt to extend it and observe the compilation error.

Program 5 – Polymorphic Shape Calculator - 4 Marks

Design an abstract class:

```
abstract class Shape
```

with:

```
abstract double area();
abstract double perimeter();
```

Create the following subclasses:

```
Circle
```

```
Rectangle
```

```
Triangle
```

Each subclass should provide its own implementation of:

```
area()
```

```
perimeter()
```

In main() create:

```
Shape s;
```

and assign different subclass objects:

```
s = new Circle(...);
```

```
s.area();
```

```
s.perimeter();
```

```
s = new Rectangle(...);
```

```
s.area();
```

```
s.perimeter();
```

```
s = new Triangle(...);
```

```
s.area();
```

```
s.perimeter();
```

Requirements

- a) Use an abstract superclass.
- b) Override both abstract methods.
- c) Use a superclass reference.
- d) Demonstrate runtime polymorphism.
- e) Display area and perimeter for each shape.

Part B – Program Analysis and Debugging - 5 Marks | BTL: L4 – Analyze

Question 1 – Analyze Abstract Class - 2 Marks

Analyze the following program:

```
abstract class Vehicle {
    abstract void start();
    void display() {
        System.out.println("Vehicle");
    }
}

class Car extends Vehicle {
    void start() {
```

```

        System.out.println("Car starts");
    }
}
public class Test {
    public static void main(String[] args) {
        Vehicle v = new Car();
        v.start();
        v.display();
    }
}

```

Tasks

- Predict the output.
- Identify which class provides the implementation of `start()`.
- Explain why `v.start()` executes the `Car` implementation.
- Identify the type of polymorphism demonstrated.

Question 2 – Debugging `final` Method - 3 Marks

The following program contains an error:

```

class Parent {
    final void display() {
        System.out.println("Parent");
    }
}
class Child extends Parent {

    void display() {
        System.out.println("Child");
    }
}

```

Tasks

- Identify the compilation error.
- Explain why the error occurs.
- Correct the program.
- Explain the purpose of the `final` keyword in this situation.

Part C – Evaluation - 5 Marks | BTL: L5 – Evaluate

Question – Abstract Class vs Concrete Class

Consider the following two designs.

Design A

```

class Shape {
    void area() {
        // general implementation
    }
}

```

Design B

```

abstract class Shape {
    abstract void area();
}

```

Suppose an application has different types of shapes such as:

- Circle
- Rectangle
- Triangle

Each shape has its own method of calculating area.

Tasks

Evaluate both designs with respect to:

- a) Abstraction
- b) Code structure
- c) Enforcing subclass implementation
- d) Extensibility
- e) Polymorphism

Then answer:

Which design would you recommend for a system containing several different types of shapes? Justify your answer with at least three technical reasons.

Part D – Viva Voce | BTL: L1–L2 (Will be assessed during PCA1/PCA2)

L1 – Remember

1. What is abstraction?
2. What is an abstract class?
3. What is an abstract method?
4. Which keyword is used to declare an abstract class?
5. Which keyword is used to declare an abstract method?
6. Can we create an object of an abstract class?
7. What is the `final` keyword?
8. What is a final variable?
9. What is a final method?
10. What is a final class?

L2 – Understand

11. Why is an abstract class used?
12. Can an abstract class contain concrete methods?
13. Can an abstract class have a constructor?
14. Why can't an abstract class be instantiated?
15. What happens if a subclass does not implement all abstract methods?
16. What is the difference between an abstract method and a concrete method?
17. Explain runtime polymorphism using an abstract class.
18. Why can an abstract class reference refer to a subclass object?
19. Can a final method be overridden?
20. Can a final class be inherited?
21. Can the value of a final variable be changed after initialization?
22. Differentiate between `abstract` and `final`.
23. What is dynamic method dispatch?
24. Why is abstraction useful in software design?
25. Give one real-world example of abstraction.

Assignment 6:

Title: Interfaces, Interface Implementation, Multiple Inheritance through Interfaces and Interface-Based Polymorphism

Duration: 1 Laboratory Session (2 Hours)

Total Marks: 30

CO Mapping: CO3

BTL Levels Covered: L3, L4, L5

Part A – Programming Exercises - 20 Marks | BTL: L3 – Apply

Students should write, compile, execute, test and demonstrate all five programs.

Program 1 – Basic Interface Implementation - 4 Marks

Create an interface `Printable` containing:

```
void print();
```

Create a class `Report` that implements the interface.

Implement the `print()` method to display:

```
Printing Report
```

Requirements

- Declare the interface.
- Implement the interface using `implements`.
- Create a `Report` object.
- Invoke the `print()` method.
- Explain why the class must provide an implementation of `print()`.

Program 2 – Interface with Constants and Multiple Implementations - 4 Marks

Create an interface `Payment` containing:

```
double SERVICE_CHARGE = 50.0;
```

```
void makePayment(double amount);
```

Create two classes:

```
CreditCardPayment
```

```
UPIPayment
```

Both classes should implement `Payment`.

Requirements

Implement `makePayment()` differently in each class.

For example:

```
Credit Card Payment
```

```
Amount: ₹5000
```

```
Service Charge: ₹50
```

```
Total: ₹5050
```

and:

```
UPI Payment
```

```
Amount: ₹5000
```

```
Service Charge: ₹50
```

```
Total: ₹5050
```

Program 3 – Implementing Multiple Interfaces - 4 Marks

Create two interfaces:

```
interface Printable {  
    void print();  
}  
interface Scannable {  
    void scan();  
}
```

Create a class:

```
class MultiFunctionPrinter implements Printable, Scannable
```

Implement:

```
print()  
scan()
```

Requirements

- Create a MultiFunctionPrinter object.
- Invoke both methods.
- Demonstrate that a Java class can implement more than one interface.
- Explain how interfaces provide a way to achieve multiple inheritance of behavior specifications.

Program 4 – Interface Reference and Polymorphism - 4 Marks

Create an interface:

```
interface Shape {  
    double area();  
}
```

Create three implementing classes:

```
Circle  
Rectangle  
Triangle
```

Each class should implement `area()` according to its own formula.

In `main()`, declare:

```
Shape s;
```

Then assign:

```
s = new Circle(...);  
System.out.println(s.area());
```

```
s = new Rectangle(...);  
System.out.println(s.area());
```

```
s = new Triangle(...);  
System.out.println(s.area());
```

Requirements

Students should demonstrate:

- Interface reference variable.
- Different implementing objects.
- Method implementation in different classes.
- Runtime polymorphism through an interface reference.

Program 5 – Multiple Interfaces and Polymorphic Design - 4 Marks

Create two interfaces:

```
interface Taxable {  
    double calculateTax();  
}  
interface Displayable {  
    void display();  
}
```

Create a class:

```
class Employee implements Taxable, Displayable
```

Data members:

```
private int empId;  
private String name;  
private double salary;
```

Implement:

```
calculateTax()  
display()
```

Use an appropriate tax calculation rule, for example:

Salary up to ₹2,50,000	→ No tax
₹2,50,001–₹5,00,000	→ 10%
Above ₹5,00,000	→ 20%

Requirements

- Implement both interfaces.
- Calculate tax.
- Display employee details.
- Use interface references where appropriate.
- Demonstrate that one class can provide implementations for multiple interfaces.

Suggested Interface References

```
Taxable t = new Employee(...);  
Displayable d = new Employee(...);
```

```
System.out.println(t.calculateTax());  
d.display();
```

Part B – Program Analysis and Debugging - 5 Marks

Question 1 – Analyze Interface Polymorphism - 2 Marks

Predict the output and explain the method invocation.

```
interface Animal {  
    void sound();  
}  
class Dog implements Animal {  
    public void sound() {  
        System.out.println("Dog barks");  
    }  
}  
class Cat implements Animal {  
    public void sound() {  
        System.out.println("Cat meows");  
    }  
}
```

```

public class Test {
    public static void main(String[] args) {
        Animal a;
        a = new Dog();
        a.sound();
        a = new Cat();
        a.sound();
    }
}

```

Tasks

- a) Predict the output.
- b) Identify the type of reference variable.
- c) Identify the actual objects.
- d) Explain why different implementations of `sound()` are executed.
- e) Identify the OOP concept demonstrated.

Question 2 – Debugging Interface Implementation - 3 Marks

The following program contains an error:

```

interface Printable {
    void print();
}
class Report implements Printable {
    void display() {
        System.out.println("Report");
    }
}
public class Test {
    public static void main(String[] args) {
        Report r = new Report();
        r.print();
    }
}

```

Tasks

1. Identify the error.
2. Explain why the program does not compile.
3. Correct the program.
4. Compile and execute the corrected program.
5. Explain what responsibility a class has when it implements an interface.

Part C – Evaluation - 5 Marks | BTL: L5 – Evaluate

Question – Abstract Class vs Interface

Consider the following two designs.

Design A – Abstract Class

```

abstract class Payment {
    abstract void makePayment();
}

```

Design B – Interface

```

interface Payment {
    void makePayment();
}

```

Suppose a banking application has different payment mechanisms:

- Credit Card
- UPI
- Net Banking
- Digital Wallet

Tasks

Evaluate **abstract class and interface** with respect to:

1. Multiple inheritance
2. Common implementation
3. Flexibility
4. Coupling
5. Reusability

Then answer:

For a system in which different and otherwise unrelated classes need to follow the same payment contract, would you recommend an abstract class or an interface? Justify your answer with at least three technical reasons.

Part D – Viva Voce | BTL: L1–L2 (Will be assessed during PCA1/PCA2)

L1 – Remember

1. What is an interface?
2. Which keyword is used to implement an interface?
3. What is an abstract method?
4. Can an interface be instantiated?
5. What is an interface reference?
6. Can a class implement multiple interfaces?
7. What is interface polymorphism?
8. What is the purpose of `implements`?
9. What is a constant in an interface?
10. Can an interface contain abstract methods?

L2 – Understand

11. Why are interfaces used?
12. Differentiate between an abstract class and an interface.
13. Can a class implement more than one interface?
14. Why does a class have to implement the methods declared by an interface?
15. Explain interface-based polymorphism.
16. What is the advantage of using an interface reference?
17. How does Java achieve multiple inheritance through interfaces?
18. Can an interface reference refer to an implementing class object?
19. Why is an interface useful for loose coupling?
20. What is the difference between `extends` and `implements`?
21. Can an interface extend another interface?
22. Can an abstract class implement an interface?
23. Why can unrelated classes implement the same interface?
24. Give a real-world example of an interface.
25. Explain the relationship between `Payment` and `CreditCardPayment`.

Assignment 7:

Title: Packages, Access Control, Java API Classes, Wrapper Classes and String Processing

Duration: 1 Laboratory Session (2 Hours)

Total Marks: 30

CO Mapping: CO3 / CO4

BTL Levels Covered: L3, L4, L5

Part A – Programming Exercises - 20 Marks | BTL: L3 – Apply

Students should write, compile, execute, test and demonstrate all five programs.

Program 1 – Creating and Using a User-Defined Package - 4 Marks

Create a package named:

`mypack`

Inside the package, create a class:

`Calculator`

Implement the following methods:

`add()`

`subtract()`

`multiply()`

`divide()`

Each method should accept two numbers and return the corresponding result.

Create another class outside the package:

`PackageDemo`

Import the `Calculator` class and use its methods.

Requirements

Students should demonstrate:

- Creating a package.
- Declaring a package using `package`.
- Importing a user-defined class.
- Creating an object of the imported class.
- Calling its methods.

Program 2 – Packages and Access Modifiers - 4 Marks

Create a package:

`college`

Inside it create:

`Student Class`

with the following members:

`public int rollNo;`

`protected String name;`

`private double marks;`

Provide appropriate methods to access the private member.

Create another class in a different package:

`test`

and demonstrate which members of `Student` can be directly accessed.

Requirements

Students should:

- Create a class in one package.

- b) Access it from another package.
- c) Test `public` access.
- d) Explain why `private` members cannot be accessed directly.
- e) Explain the role of `protected` members across packages.
- f) Use a getter/setter where required.

Program 3 – Wrapper Classes, Autoboxing and Unboxing - 4 Marks

Write a Java program demonstrating wrapper classes.

Use:

`Integer`
`Double`
`Character`
`Boolean`

The program should:

- a) Convert primitive values into wrapper objects.
- b) Demonstrate autoboxing.
- c) Demonstrate unboxing.
- d) Convert a numeric String into an `Integer`.
- e) Perform arithmetic using the extracted primitive values.

Example

```
int a = 25;
Integer obj = a;           // Autoboxing
int b = obj;               // Unboxing
String s = "100";
Integer value = Integer.parseInt(s);
```

Requirements

Display:

Primitive value
Wrapper object
Autoboxed value
Unboxed value
String converted to integer

Program 4 – String Processing - 4 Marks

Write a Java program to accept a String and perform the following operations:

- a) Display its length.
- b) Count the number of vowels.
- c) Count the number of consonants.
- d) Reverse the String.
- e) Check whether the String is a palindrome.
- f) Convert the String to uppercase.
- g) Convert the String to lowercase.

Example

Input:

MADAM

Output:

Length : 5
Vowels : 2

Consonants : 3
Reverse : MADAM
Palindrome : Yes
Uppercase : MADAM
Lowercase : madam

Program 5 – String Comparison and Searching - 4 Marks

Write a Java program to accept two strings and perform:

- Equality check using `equals()`.
- Case-insensitive comparison using `equalsIgnoreCase()`.
- Lexicographical comparison using `compareTo()`.
- Check whether one String contains another String.
- Find the position of a specified substring using `indexOf()`.
- Replace a specified word using `replace()`.

Example

Input:

String 1: Object Oriented Programming
String 2: object oriented programming

The program should demonstrate the difference between:

`equals()`

and

`equalsIgnoreCase()`

Part B – Program Analysis and Debugging - 5 Marks | BTL: L4 – Analyze

Question 1 – Package and Access Analysis - 2 Marks

Consider:

```
package p1;
public class Test {
    private int a = 10;
    public int b = 20;
    protected int c = 30;
}
```

Another class belongs to package p2.

Answer:

- Which members can be directly accessed from p2?
- Which members cannot be directly accessed?
- Why is a inaccessible?
- What is the role of public and protected here?

Students must justify their answers based on Java access rules.

Question 2 – String Analysis and Debugging - 3 Marks

Analyze the following program:

```
public class Test {
    public static void main(String[] args) {
        String s1 = "Java";
        String s2 = new String("Java");
        System.out.println(s1 == s2);
        System.out.println(s1.equals(s2));
    }
}
```

```
}
```

Tasks

- Predict the output.
- Explain why the two results are different.
- Identify which comparison should normally be used to compare String contents.
- Explain the difference between `==` and `equals()` for objects.

Part C – Evaluation - 5 Marks | BTL: L5 – Evaluate

Question – String vs StringBuilder

Consider the following two approaches for repeatedly modifying a String.

Approach A

```
String s = "";
for (int i = 1; i <= 1000; i++) {
    s = s + i;
}
```

Approach B

```
StringBuilder sb = new StringBuilder();
for (int i = 1; i <= 1000; i++) {
    sb.append(i);
}
```

Tasks

Evaluate the two approaches with respect to:

- Mutability
- Object creation
- Memory usage
- Performance
- Suitability for repeated modifications

Then answer:

For an application that performs a large number of String modifications, which approach would you recommend? Justify your answer with at least three technical reasons.

Part D – Viva Voce | BTL: L1–L2 (Will be assessed during PCA1/PCA2)

L1 – Remember

- What is a package?
- Which keyword is used to declare a package?
- Which keyword is used to import a package/class?
- What is a user-defined package?
- What is a Java API?
- What is a wrapper class?
- What is autoboxing?
- What is unboxing?
- What is a String?
- What is the difference between `String` and `StringBuilder`?

L2 – Understand

- Why are packages used?
- What are the advantages of packages?
- Explain the difference between `package` and `import`.
- Can a private member be accessed from another package?

15. Explain the role of access modifiers with packages.
16. What is autoboxing? Give an example.
17. What is unboxing?
18. Why are wrapper classes required?
19. Explain the difference between `==` and `equals()` for Strings.
20. Why is String immutable?
21. What is the purpose of `StringBuilder`?
22. Differentiate between `String` and `StringBuilder`.
23. What is `indexOf()`?
24. What is `substring()`?
25. What is `compareTo()`?

Assignment 8:

Title: Exception Handling in Java: Built-in Exceptions, Multiple Catch, finally, throw, throws and User-Defined Exceptions

Duration: 1 Laboratory Session (2 Hours)

Total Marks: 30

CO Mapping: CO4

BTL Levels Covered: L3, L4, L5

Part A – Programming Exercises - 20 Marks | BTL: L3 – Apply

Students should write, compile, execute, test and demonstrate all five programs.

Program 1 – Handling a Built-in Exception - 4 Marks

Write a Java program to accept two integers and perform division.

The program should handle:

`ArithmeticException`

when the denominator is zero.

Requirements

- Accept numerator and denominator.
- Perform division inside a `try` block.
- Handle `ArithmeticException` using `catch`.
- Display an appropriate error message.
- Demonstrate that the program continues normally after exception handling.

Sample Output

For:

Enter numerator: 20

Enter denominator: 0

Output:

Cannot divide by zero.

Program continues normally.

Program 2 – Multiple Catch Blocks - 4 Marks

Write a Java program to accept an integer `n` and create an integer array of size `n`.

Accept an array index from the user and display the corresponding element.

The program should handle:

- `ArithmeticException`
- `ArrayIndexOutOfBoundsException`
- `NumberFormatException`

using appropriate `catch` blocks.

Requirements

Demonstrate different types of invalid input and show an appropriate message for each exception.

For example:

Enter array size: 5

Enter index: 10

Output:

Error: Array index is outside the valid range.

Program 3 – finally and Nested try - 4 Marks

Write a Java program to demonstrate the use of `finally`.

The program should:

- a) Accept two numbers.
- b) Perform division.
- c) Handle division by zero.
- d) Use a `finally` block to display:

Execution of calculation completed.

Then modify the program to demonstrate a nested `try` block.

Requirements

Students should demonstrate that the `finally` block executes whether an exception occurs or not.

Example

```
try {
    // calculation
    try {
        // inner operation
    }
    catch (...) {
        // handle inner exception
    }
}
catch (...) {
    // handle outer exception
}
finally {
    System.out.println("Execution completed.");
}
```

Program 4 – throw and throws - 4 Marks

Create a class `Student` with:

`rollNo`
`name`
`marks`

Create a method:

`void validateMarks() throws Exception`

The method should check whether the marks are within the range:

0 to 100

If the marks are invalid, explicitly generate an exception using:

`throw new Exception("Invalid marks");`

Requirements

1. Use `throw` to explicitly generate an exception.
2. Use `throws` in the method declaration.
3. Handle the exception in the calling method.
4. Display an appropriate message.

Example

Enter marks: 125

Error: Invalid marks.

Marks must be between 0 and 100.

Program 5 – User-Defined Exception - 4 Marks

Create a user-defined exception:

```
class InvalidAgeException extends Exception
```

Create a class `Voter` containing:

```
name
```

```
age
```

Create a method:

```
void validateAge() throws InvalidAgeException
```

The method should throw `InvalidAgeException` if the age is less than 18.

Requirements

1. Create a user-defined exception.
2. Extend the `Exception` class.
3. Use `throw`.
4. Declare the exception using `throws`.
5. Handle it using `try-catch`.
6. Display an appropriate message.

Example

```
Enter name: Rahul
```

```
Enter age: 16
```

```
Exception: Rahul is not eligible for voting.
```

Part B – Program Analysis and Debugging - 5 Marks | BTL: L4 – Analyze

Question 1 – Analyze Exception Flow - 2 Marks

Predict the output:

```
public class Test {  
    public static void main(String[] args) {  
        try {  
            int a = 10;  
            int b = 0;  
            System.out.println(a / b);  
        }  
        catch (ArithmeticException e) {  
            System.out.println("Exception handled");  
        }  
        finally {  
            System.out.println("Finally executed");  
        }  
        System.out.println("Program continues");  
    }  
}
```

Tasks

1. Predict the complete output.
2. Identify which statement causes the exception.
3. Explain why the statements after the exception inside `try` are not executed.
4. Explain why `finally` executes.

Expected Output

```
Exception handled
```

```
Finally executed
```

```
Program continues
```

Question 2 – Debugging Exception Handling - 3 Marks

The following program contains an error:

```
public class Test {  
    public static void main(String[] args) {  
        try {  
            int a = 10;  
            int b = 0;  
            System.out.println(a / b);  
        }  
        System.out.println("Calculation completed");  
        catch (ArithmeticException e) {  
            System.out.println("Cannot divide by zero");  
        }  
    }  
}
```

Tasks

1. Identify the syntax error.
2. Explain why the error occurs.
3. Rewrite the corrected program.
4. Compile and execute it.
5. Explain the correct position of statements relative to `try` and `catch`.

Expected Concept

The statement:

```
System.out.println("Calculation completed");
```

cannot be placed between the `try` block and its corresponding `catch` block.

Part C – Evaluation - 5 Marks | BTL: L5 – Evaluate

Question – Exception Handling Design

Consider the following two approaches.

Approach A

```
int divide(int a, int b) {  
    if (b == 0) {  
        return 0;  
    }  
    return a / b;  
}
```

Approach B

```
int divide(int a, int b) throws ArithmeticException {  
    if (b == 0) {  
        throw new ArithmeticException("Division by zero");  
    }  
    return a / b;  
}
```

Tasks

Evaluate both approaches with respect to:

- a) Error identification
- b) Error reporting
- c) Program reliability
- d) Separation of normal and exceptional conditions
- e) Maintainability

Then answer:

For a large software application where invalid input must be clearly distinguished from a valid result, which approach would you recommend? Justify your answer with at least three technical reasons.

Part D – Viva Voce | BTL: L1–L2 (Will be assessed during PCA1/PCA2)

L1 – Remember

1. What is an exception?
2. What is exception handling?
3. Which keywords are used for exception handling in Java?
4. What is `try`?
5. What is `catch`?
6. What is `finally`?
7. What is `throw`?
8. What is `throws`?
9. What is a user-defined exception?
10. Name two predefined exceptions.

L2 – Understand

11. Why is exception handling required?
12. What happens when an exception occurs inside a `try` block?
13. What is the purpose of a `catch` block?
14. Why is `finally` used?
15. Differentiate between `throw` and `throws`.
16. What is exception propagation?
17. What is the difference between checked and unchecked exceptions?
18. Explain `ArithmeticException`.
19. Explain `ArrayIndexOutOfBoundsException`.
20. Why should specific exceptions generally be handled appropriately?
21. Can we have multiple `catch` blocks?
22. Can `try` exist without `catch`?
23. Can `finally` be used with `try`?
24. What is a user-defined exception?
25. Why would we create a user-defined exception instead of using a predefined exception?

Assignment 9:

Title: Multithreading in Java: Thread Creation, Thread Life Cycle, Thread Methods, Multiple Threads and Synchronization

Duration: 1 Laboratory Session (2 Hours)

Total Marks: 30

CO Mapping: CO4

BTL Levels Covered: L3, L4, L5

Part A – Programming Exercises - 20 Marks | BTL: L3 – Apply

Students should write, compile, execute, test and demonstrate all five programs.

Program 1 – Creating a Thread by Extending Thread - 4 Marks

Create a class `MyThread` by extending the `Thread` class.

Override the `run()` method to display the numbers from **1 to 10**.

Create the thread object in the `main()` method and start the thread using:

```
start();
```

Requirements

- Extend the `Thread` class.
- Override `run()`.
- Create a thread object.
- Invoke `start()`.
- Display numbers from 1 to 10.
- Explain the difference between calling `run()` and `start()`.

Program 2 – Creating a Thread Using Runnable - 4 Marks

Create a class `NumberPrinter` that implements the `Runnable` interface.

The `run()` method should display the first **10 even numbers**.

Create a `Thread` object using the `NumberPrinter` object and start the thread.

Requirements

Demonstrate:

```
Runnable r = new NumberPrinter();  
Thread t = new Thread(r);  
t.start();
```

The program should display:

2, 4, 6, 8, 10, 12, 14, 16, 18, 20

Students should explain

- Why `Runnable` is used.
- Why the `Thread` object is required.
- Difference between extending `Thread` and implementing `Runnable`.

Program 3 – Multiple Threads with `sleep()`, `isAlive()` and `join()` - 4 Marks

Create two threads:

Thread 1 – `EvenThread`

Display the first 10 even numbers.

Thread 2 – `OddThread`

Display the first 10 odd numbers.

Use: `sleep()` to introduce a small delay between displaying numbers.

Use: `isAlive()` to check whether a thread is running.

Use: `join()` to make the main thread wait for completion.

Requirements

The program should demonstrate:

```
t1.start();
```

```
t2.start();
```

```
System.out.println(t1.isAlive());
```

```
t1.join();
```

```
t2.join();
```

```
System.out.println("Both threads completed.");
```

Important Observation

Students should note that the order in which the even and odd numbers appear may vary because the threads execute concurrently.

Program 4 – Multithreaded Task Processing - 4 Marks

Create two threads to perform two independent tasks.

Thread 1 – FactorialThread

Calculate the factorial of a given number.

Thread 2 – PrimeThread

Check whether another given number is prime.

For example:

Input:

Number for factorial = 5

Number for prime check = 29

Possible output:

Factorial of 5 = 120

29 is a prime number

Requirements

- Create two separate threads.
- Each thread should perform a different task.
- Start both threads.
- Use `join()` in the main method.
- Display the final message after both threads finish.

Program 5 – Synchronization and Shared Resource - 4 Marks

Create a class:

```
class BankAccount
```

```
with: private int balance = 1000;
```

Create a method: `withdraw(int amount)`

Two different threads should attempt to withdraw money from the same account.

For example:

Initial Balance = ₹1000

Thread 1 attempts to withdraw ₹700

Thread 2 attempts to withdraw ₹500

Without synchronization, the program may produce an inconsistent result depending on execution.

Modify the method using:

`synchronized`

so that only one thread can access the critical section at a time.

Requirements

- a) Create a shared resource.
- b) Create two threads.
- c) Make both threads access the same account.
- d) Demonstrate the need for synchronization.
- e) Use a synchronized method or synchronized block.
- f) Display the final balance.

Part B – Program Analysis and Debugging - 5 Marks | BTL: L4 – Analyze

Question 1 – `run()` vs `start()` – 2 Marks

Analyze the following program:

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread is running");
    }
    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.run();
        System.out.println("Main method completed");
    }
}
```

Tasks

- a) Predict the output.
- b) Does this program actually create a new thread of execution?
- c) Explain what happens when `run()` is called directly.
- d) Rewrite the program using `start()`.

Expected Output

Thread is running

Main method completed

Important Concept

Calling: `t.run()` does **not** start a new thread.

Calling: `t.start()` requests the JVM to start a new thread, which then executes `run()`.

Question 2 – Analyze Thread Execution - 3 Marks

Consider:

```
class Test extends Thread {
    public void run() {
        for(int i = 1; i <= 3; i++) {
            System.out.println("Child Thread: " + i);
            try {
                Thread.sleep(100);
            }
            catch (InterruptedException e) {
                System.out.println(e);
            }
        }
    }
}
```

```

        }
    }
    public static void main(String[] args) {
        Test t = new Test();
        t.start();
        for(int i = 1; i <= 3; i++) {
            System.out.println("Main Thread: " + i);
        }
    }
}

```

Tasks

- Is the exact output order guaranteed?
- Explain why the output order can vary.
- Identify the two threads involved.
- Explain the effect of `sleep()`.
- Suggest how `join()` could be used if the main thread must wait for the child thread.

Part C – Evaluation - 5 Marks | BTL: L5 – Evaluate

Question – Thread vs Runnable

A developer wants to create multiple threads in a Java application.

Approach A

```

class Task extends Thread {
    public void run() {
        // task
    }
}

```

Approach B

```

class Task implements Runnable {
    public void run() {
        // task
    }
}

```

Task

Evaluate both approaches considering:

- Inheritance limitation
- Code reusability
- Separation of task and thread
- Flexibility
- Suitability for larger applications

Then answer:

Which approach would you recommend for developing a larger Java application containing several independent tasks? Justify your answer with at least three technical reasons.

Part D – Viva Voce | BTL: L1–L2 (Will be assessed during PCA1/PCA2)

L1 – Remember

- What is a thread?
- What is multithreading?
- Which class is used to create a thread?
- Which interface can be implemented to create a thread?

5. What is the purpose of `run()`?
6. What is the purpose of `start()`?
7. What is `sleep()`?
8. What is `join()`?
9. What is `isAlive()`?
10. What is synchronization?

L2 – Understand

11. Differentiate between a process and a thread.
12. Differentiate between `run()` and `start()`.
13. Why is `Runnable` used?
14. Explain the thread life cycle.
15. Why does the order of execution of multiple threads vary?
16. What is the purpose of `sleep()`?
17. What happens when `join()` is called?
18. What does `isAlive()` return?
19. What is a race condition?
20. Why is synchronization required?
21. Explain the purpose of the `synchronized` keyword.
22. Differentiate between extending `Thread` and implementing `Runnable`.
23. Can two threads access the same object?
24. What is a shared resource?
25. Give a real-world application of multithreading.

Assignment 10:

Title: Java Collections Framework: ArrayList, LinkedList, HashSet, HashMap and Iterators

Duration: 1 Laboratory Session (2 Hours)

Total Marks: 30

CO Mapping: CO4

BTL Levels Covered: L3, L4, L5

Part A – Programming Exercises - 20 Marks | BTL: L3 – Apply

Students should write, compile, execute, test and demonstrate all five programs.

Program 1 – ArrayList Operations - 4 Marks

Create an `ArrayList<Integer>` to store the marks of students.

The program should provide the following operations:

- a) Add marks.
- b) Display all marks.
- c) Insert a mark at a specified position.
- d) Remove a mark.
- e) Search for a particular mark.
- f) Display the total number of elements.

Students should use

```
add()  
get()  
set()  
remove()  
contains()  
size()
```

Program 2 – LinkedList for Student Records - 4 Marks

Create a class:

```
Student
```

with:

```
rollNo  
name  
marks
```

Create a:

```
LinkedList<Student> to store student objects.
```

Implement the following operations:

- a) Add student.
- b) Display all students.
- c) Add a student at the beginning.
- d) Add a student at the end.
- e) Remove a student based on roll number.
- f) Search for a student based on roll number.

Program 3 – Removing Duplicate Elements Using HashSet - 4 Marks

Write a Java program to accept a list of integers containing duplicate values.

Store the values in a:

`HashSet<Integer>` and display the unique values.

Example

Input:

10 20 10 30 20 40 30 50

Output:

Unique elements:

10 20 30 40 50

Requirements

- Accept at least 10 integers.
- Store them in a `HashSet`.
- Display the unique elements.
- Display the size of the original collection and the set.
- Explain why duplicate elements are not stored in a `HashSet`.

Program 4 – HashMap for Student Marks - 4 Marks

Create a `HashMap<Integer, String>` to store:

Roll Number → Student Name

Perform the following operations:

- Add student records.
- Display all records.
- Search for a student using roll number.
- Update a student's name.
- Remove a student.
- Check whether a particular roll number exists.

Then create another:

`HashMap<Integer, Double>` to store: Roll Number → Marks and display the marks of a specified student.

Required Methods

Students should demonstrate:

```
put ()
get ()
containsKey ()
remove ()
replace ()
size ()
```

Program 5 – Iterator-Based Collection Processing - 4 Marks

Create an `ArrayList<String>` containing the names of at least 10 students.

Use an: `Iterator<String>` to traverse and display the elements.

Modify the program so that students whose names satisfy a specified condition are removed using the iterator.

For example: Remove all names having fewer than 5 characters.

Requirements

Students should demonstrate:

```
Iterator<String> itr = list.iterator();
while(itr.hasNext()) {
    String name = itr.next();
    ...
}
```

```
}
```

and, where applicable: `itr.remove()`;

Part B – Program Analysis and Debugging - 5 Marks | BTL: L4 – Analyze

Question 1 – ArrayList vs HashSet Analysis - 2 Marks

Consider:

```
ArrayList<Integer> list = new ArrayList<>();  
list.add(10);  
list.add(20);  
list.add(10);  
list.add(30);  
list.add(20);
```

```
System.out.println(list);
```

and:

```
HashSet<Integer> set = new HashSet<>();
```

```
set.add(10);  
set.add(20);  
set.add(10);  
set.add(30);  
set.add(20);
```

```
System.out.println(set);
```

Tasks

- How many elements will the `ArrayList` contain?
- How many elements will the `HashSet` contain?
- Why are duplicate values treated differently?
- Which collection would you choose if duplicate values must be eliminated?

Expected Observation

`ArrayList` permits duplicate elements, whereas `HashSet` stores unique elements.

Question 2 – Debugging Iterator Usage - 3 Marks

Identify the problem in the following program:

```
import java.util.*;  
class Test {  
    public static void main(String[] args) {  
        ArrayList<Integer> list = new ArrayList<>();  
        list.add(10);  
        list.add(20);  
        list.add(30);  
        list.add(40);  
        Iterator<Integer> itr = list.iterator();  
        while(itr.hasNext()) {  
            Integer x = itr.next();  
            if(x == 20) {  
                list.remove(x);  
            }  
        }  
        System.out.println(list);  
    }  
}
```

Tasks

- a) Identify the problem.
- b) Explain why directly modifying the collection while using an iterator can cause an exception.
- c) Rewrite the program using the iterator correctly.
- d) Execute the corrected program.
- e) Explain the purpose of `Iterator.remove()`.

Expected Concept

When an element must be removed during iterator traversal, use: `itr.remove()` ;
rather than directly modifying the collection.

Part C – Evaluation - 5 Marks | BTL: L5 – Evaluate

Question – Choosing the Appropriate Collection

Consider the following requirements:

Situation A

A student database needs to maintain records in insertion order and allow duplicate entries.

Situation B

A system needs to store unique student roll numbers.

Situation C

A system needs to store:

Roll Number → Student Name

and retrieve the name quickly using the roll number.

Situation D

A program frequently inserts and removes elements from a list.

Task

Evaluate the following collections:

- `ArrayList`
- `LinkedList`
- `HashSet`
- `HashMap`

and select the most appropriate collection for each situation.

Part D – Viva Voce | BTL: L1–L2 (Will be assessed during PCA1/PCA2)

L1 – Remember

1. What is the Collection Framework?
2. What is a collection?
3. What is `ArrayList`?
4. What is `LinkedList`?
5. What is `HashSet`?
6. What is `HashMap`?
7. What is an `Iterator`?
8. Which collection allows duplicate elements?
9. Which collection stores key-value pairs?
10. Which collection does not allow duplicate elements?

L2 – Understand

11. Differentiate between `ArrayList` and `LinkedList`.
12. Differentiate between `ArrayList` and `HashSet`.

13. Why does `HashSet` not allow duplicate elements?
14. Explain the key-value concept of `HashMap`.
15. Why are generics used in collections?
16. What is the purpose of `Iterator`?
17. What is the difference between `hasNext()` and `next()`?
18. Why should `Iterator.remove()` be used while removing an element during traversal?
19. Can a `HashMap` contain duplicate keys?
20. Can a `HashMap` contain duplicate values?
21. What happens when an existing key is inserted again into a `HashMap`?
22. Why would you choose `LinkedList` over `ArrayList`?
23. Why would you choose `HashSet` instead of `ArrayList`?
24. Give a real-world application of `HashMap`.
25. Give a real-world application of `HashSet`.

Assignment 11:

Title: Java Swing GUI Programming and Event-Driven Application Development

Duration: 1 Laboratory Session (2 Hours)

Total Marks: 30

CO Mapping: CO5

BTL Levels Covered: L3, L4, L5, L6

Part A – Programming / GUI Development - 20 Marks | BTL: L3–L6

Students should write, compile, execute, test and demonstrate all five programs.

Program 1 – Basic Swing Form - 4 Marks

Create a Java Swing application to design a Student Information Form.

The form should contain:

- `JFrame`
- `JLabel`
- `JTextField`
- `JButton`

Form Fields

Roll Number:

Student Name:

Department:

Include buttons:

Submit

Clear

When the user clicks **Submit**, display the entered information using `JOptionPane`.

When the user clicks **Clear**, clear all text fields.

Program 2 – Calculator Using Swing - 4 Marks

Develop a simple **Swing Calculator**.

The GUI should contain:

Number 1: _____

Number 2: _____

[Add] [Subtract] [Multiply] [Divide]

Result: _____

Requirements

1. Accept two numbers.
2. Implement:
 - Addition
 - Subtraction
 - Multiplication
 - Division
3. Display the result.
4. Handle division by zero appropriately.
5. Provide a **Clear** button.

Program 3 – Student Registration Form - 4 Marks | BTL: L4 – Analyze / Apply

Design a Student Registration Form using:

- JTextField
- JRadioButton
- JCheckBox
- JComboBox
- JButton
- JTextArea

Required Fields

Name:

Roll Number:

Gender: Male / Female / Other

Department:

CSE
CSE (AI/ML)
ECE
EE
EEE
ME

Programming Skills:

Java
Python
C++
JavaScript

[Submit] [Reset]

On clicking **Submit**, display all selected information in a JTextArea.

Requirements

Students must:

- a) Group the radio buttons using `ButtonGroup`.
- b) Allow multiple programming skills through checkboxes.
- c) Select one department through `JComboBox`.
- d) Display the complete information.
- e) Implement Reset functionality.

Program 4 – GUI Application with Validation - 4 Marks | BTL: L4 – Analyze

Develop a Student Marks and Grade Calculator using Swing.

Input

Student Name: _____
Roll Number: _____
Marks in Java: _____
Marks in Data Structure: _____
Marks in Mathematics: _____
[Calculate]

The application should calculate:

Total Marks
Average
Grade

Use the following grading system:

Average Grade

90–100 A+
80–89 A
70–79 B
60–69 C
50–59 D
Below 50 F

Validation

The program should display an appropriate message if:

- a) A field is empty.
- b) Marks are not numeric.
- c) Marks are outside 0–100.

Program 5 – Design and Develop a Mini GUI Application - 4 Marks | BTL: L6 – Create

Library Book Management Form

Design and develop a small **Library Book Management System** using Java Swing.

The application should allow the user to:

- a) Enter Book ID.
- b) Enter Book Name.
- c) Enter Author Name.
- d) Select Book Category.
- e) Select availability status.
- f) Add the book.
- g) Display the entered book information.
- h) Clear the form.

Suggested GUI

```
LIBRARY BOOK MANAGEMENT
-----
Book ID       : [_____]
Book Name     : [_____]
Author        : [_____]
Category      : [_____] ▼
Availability   : ☐ Available
```

o Issued
[Add Book] [Clear]

Book Details

Book ID :
Book Name :
Author :
Category :
Availability :

Minimum Components

The application should use:

- JFrame
- JPanel
- JLabel
- JTextField
- JComboBox
- JRadioButton
- ButtonGroup
- JButton
- JTextArea

Requirements

Students must:

- a) Design the GUI layout.
- b) Select appropriate Swing components.
- c) Implement event handling.
- d) Validate user input.
- e) Display the entered information.
- f) Implement Clear functionality.
- g) Use meaningful component names.
- h) Demonstrate the working application.

Part B – Program Analysis and Debugging - 5 Marks | BTL: L4 – Analyze

Question 1 – Event Handling Analysis - 2 Marks

Analyze the following program:

```
import javax.swing.*;
import java.awt.event.*;
class Test extends JFrame implements ActionListener {
    JButton button;
    Test() {
        button = new JButton("Click");
        add(button);
        button.addActionListener(this);
        setSize(300,200);
        setVisible(true);
    }

    public void actionPerformed(ActionEvent e) {
        JOptionPane.showMessageDialog(this,
            "Button Clicked");
    }
}
```

```

    public static void main(String[] args) {
        new Test();
    }
}

```

Tasks

- What happens when the button is clicked?
- What is the purpose of `ActionListener`?
- Why is `addActionListener()` required?
- What role does `actionPerformed()` play?
- Identify the event source.

Expected Concept

Students should identify:

`JButton` → Event Source

`ActionListener` → Event Listener

`actionPerformed()` → Event Handling Method

Question 2 – Debugging Swing Code - 3 Marks

The following program contains errors:

```

import javax.swing.*;
class Test extends JFrame {
    JButton button;
    Test() {
        button = new JButton("Submit");
        add(button);
        button.addActionListener();
        setSize(300,200);
        setVisible(true);
    }
}

```

Tasks

- Identify the error.
- Explain why the error occurs.
- Rewrite the corrected program.
- Add an appropriate event listener.
- Execute and verify the application.

Expected Concept

`addActionListener()` requires an appropriate `ActionListener` object.

Part C – Evaluation and Design - 5 Marks | BTL: L5–L6

Question – Evaluate and Design a GUI

Consider two approaches to GUI development.

Approach A

Create a GUI containing many components without considering:

- component grouping,
- layout,
- validation,
- usability,
- event handling.

Approach B

Design the GUI systematically by:

1. Identifying user requirements.
2. Selecting appropriate components.
3. Arranging components using suitable layouts.
4. Implementing event handling.
5. Validating input.
6. Providing meaningful feedback.
7. Testing different user interactions.

Tasks

Part 1 – Evaluation – 3 Marks

Evaluate both approaches with respect to:

- usability,
- maintainability,
- reliability.

Part 2 – Design – 2 Marks

Design a GUI for Student Attendance Management containing:

Student Name

Roll Number

Department

Semester

Attendance %

Status

and buttons:

Calculate Status

Clear

Exit

Students should draw the GUI layout and explain:

- a) Which Swing components they would use.
- b) Which events they would handle.
- c) How they would validate the attendance percentage.

Part D – Viva Voce | BTL: L1–L2 (Will be assessed during PCA1/PCA2)

L1 – Remember

1. What is Swing?
2. What is JFrame?
3. What is JLabel?
4. What is JTextField?
5. What is JButton?
6. What is JCheckBox?
7. What is JRadioButton?
8. What is JComboBox?
9. What is JTextArea?
10. What is ActionListener?
11. What is ActionEvent?
12. What is JOptionPane?

L2 – Understand

13. What is GUI programming?

14. What is event-driven programming?
15. Why is `JFrame` used?
16. What is the difference between `JTextField` and `JTextArea`?
17. Why is `ButtonGroup` used with radio buttons?
18. Differentiate between `JCheckBox` and `JRadioButton`.
19. Why is `ActionListener` required for a button?
20. What happens when a `JButton` is clicked?
21. What is the purpose of `actionPerformed()`?
22. Why are layout managers used?
23. What is the difference between a component and a container?
24. Why should GUI input be validated?
25. Give two real-world applications of Java Swing.

Assignment 12:

Title: Model–View–Controller (MVC) Pattern and Command Design Pattern in Java

Duration: 1 Laboratory Session (2 Hours)

Total Marks: 30

CO Mapping: CO5

BTL Levels Covered: L3, L4, L5, L6

Part A – Programming / Implementation - 20 Marks | BTL: L3–L6

Students should write, compile, execute, test and demonstrate all five programs.

Program 1 – Basic MVC Implementation - 4 Marks

Develop a simple Java application using the **MVC pattern** to maintain student information.

Model

Create:

```
class Student
```

with:

```
private int rollNo;  
private String name;  
private double marks;
```

Provide appropriate constructors, getters and setters.

View

Create a class:

```
class StudentView with methods to display student information.
```

For example: `displayStudentDetails()`

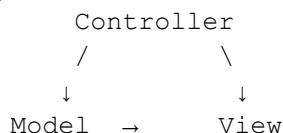
Controller

Create: `class StudentController` which connects the Model and View.

Main

Create a student object and use the controller to update and display student information.

Required Flow



Requirements

- a) Create separate Model, View and Controller classes.

- b) Store student information in Model.
- c) Display information through View.
- d) Use Controller to modify/access Model.
- e) Demonstrate separation of responsibilities.

Program 2 – MVC-Based Calculator - 4 Marks | BTL: L3 – Apply

Develop a Calculator application using MVC architecture.

Model

Create:

```
class CalculatorModel
```

with methods:

```
add()  
subtract()  
multiply()  
divide()
```

View

Create:

```
class CalculatorView
```

to accept/display:

```
Number 1  
Number 2  
Result
```

Controller

Create:

```
class CalculatorController
```

to receive requests from the View and invoke appropriate Model operations.

Example

```
Number 1 : 20  
Number 2 : 5  
Addition      : 25  
Subtraction   : 15  
Multiplication : 100  
Division      : 4
```

Requirements

Students must ensure that:

- a) Model performs calculations.
- b) View handles presentation.
- c) Controller coordinates Model and View.

Program 3 – Command as Methods - 4 Marks | BTL: L4 – Analyze

Consider a simple Light class:

```
class Light {  
    void switchOn() {  
        System.out.println("Light ON");  
    }  
    void switchOff() {  
        System.out.println("Light OFF");  
    }  
}
```

Create a Java program where commands are represented directly as methods.

The user should be able to perform:

1. Switch ON
2. Switch OFF
3. Exit

Example

Enter command: 1

Light ON

Enter command: 2

Light OFF

Students should analyze

- a) How is the request represented?
- b) Which object receives the request?
- c) Which method executes the request?
- d) What happens if the number of commands increases?
- e) What limitations exist when commands are directly implemented as methods?

Program 4 – Command as Objects - 4 Marks | BTL: L4 – Analyze / Apply

Redesign Program 3 using the Command design pattern.

Create an interface:

```
interface Command {  
    void execute();  
}
```

Create a receiver:

```
class Light {  
    void switchOn() {  
        System.out.println("Light ON");  
    }  
    void switchOff() {  
        System.out.println("Light OFF");  
    }  
}
```

Create concrete command classes:

```
class LightOnCommand implements Command
```

and:

```
class LightOffCommand implements Command
```

Create an Invoker:

```
class RemoteControl
```

with an appropriate method to execute the command.

Required Architecture

Client

↓

Invoker

↓

Command Interface

+----- LightOnCommand

+----- LightOffCommand

↓

Receiver

Light

Demonstrate

```
remote.setCommand(onCommand);
```

```
remote.pressButton();
remote.setCommand(offCommand);
remote.pressButton();
```

Students should explain

- What is the Command object?
- What is the Receiver?
- What is the Invoker?
- What is the Client?
- Why is the request represented as an object?

Program 5 – Mini MVC + Command Application - 4 Marks | BTL: L6 – Create Student Task Management System

Design and develop a small Java application that combines:

- MVC architecture
- Command pattern

The system should manage student tasks.

Model

Create: `class Task`

containing:

```
taskId
taskName
status
```

Possible status:

```
PENDING
COMPLETED
```

View

Create: `class TaskView` to display the task details.

Controller

Create: `class TaskController` to coordinate the Model and View.

Commands

Create a Command interface:

```
interface Command {
    void execute();
}
```

Create command objects such as:

```
AddTaskCommand
CompleteTaskCommand
DisplayTaskCommand
```

Invoker

Create: `class TaskManager` which accepts and executes command objects.

Example Interaction

```
===== STUDENT TASK MANAGER =====
```

- Add Task
- Complete Task
- Display Tasks
- Exit

Enter choice: 1

Task ID: 101

Task Name: Complete Java Assignment

Task Added Successfully

Then:

Enter choice: 2

Enter Task ID: 101

Task marked as COMPLETED.

Requirements

Students must:

- a) Separate Model, View and Controller.
- b) Define a Command interface.
- c) Implement at least three concrete commands.
- d) Create an Invoker.
- e) Use objects to represent commands.
- f) Demonstrate command execution.
- g) Maintain proper encapsulation.
- h) Test all operations.

Part B – Program Analysis and Debugging - 5 Marks | BTL: L4 – Analyze

Question 1 – MVC Analysis - 2 Marks

Consider the following code:

```
class Student {  
    String name;  
    int marks;  
    void display() {  
        System.out.println(name);  
        System.out.println(marks);  
    }  
}
```

The same class also accepts keyboard input, calculates marks and displays the output.

Tasks

- a) Identify the design problem.
- b) Which MVC components are mixed together?
- c) Explain why separating Model, View and Controller can improve the design.

Question 2 – Command Pattern Analysis - 3 Marks

Consider:

```
class RemoteControl {  
    Light light;  
    void pressButton(int choice) {  
        if(choice == 1)  
            light.switchOn();  
        else if(choice == 2)  
            light.switchOff();  
        else if(choice == 3)  
            light.increaseBrightness();  
    }  
}
```

Tasks

- a) Identify the design limitation of this approach.
- b) What happens if 10 more commands are added?

- c) Explain how the Command pattern can improve this design.
- d) Identify the possible:
 - a. Invoker
 - b. Command
 - c. Concrete Command
 - d. Receiver
- e) Suggest how polymorphism can be used to remove the large `if-else` structure.

Part C – Evaluation and Design - 5 Marks | BTL: L5–L6

Question – Evaluate MVC and Command-Based Designs

A developer has created two versions of a Student Management application.

Version A

All operations are written in a single class:

Input
Processing
Business Logic
Output
Command Execution

Version B

The application uses:

Model
View
Controller
Command
Invoker
Receiver

Task 1 – Evaluation

Compare Version A and Version B with respect to:

1. Maintainability
2. Reusability
3. Extensibility
4. Testing
5. Separation of concerns

Task 2 – Design

Design the class structure for a **Library Management System** using MVC and Command patterns.

The system should support:

Add Book
Issue Book
Return Book
Display Book

Students should identify:

- Model
- View
- Controller
- Command interface
- Concrete Commands
- Invoker
- Receiver

Part D – Viva Voce | BTL: L1–L2 (Will be assessed during PCA1/PCA2)

L1 – Remember

1. What is a design pattern?
2. What is MVC?
3. What does MVC stand for?
4. What is the Model?
5. What is the View?
6. What is the Controller?
7. What is the Command pattern?
8. What is a Command object?
9. What is an Invoker?
10. What is a Receiver?

L2 – Understand

11. Why is MVC used?
12. Explain the relationship between Model, View and Controller.
13. What is separation of concerns?
14. What is the difference between a command as a method and a command as an object?
15. Why is a Command interface used?
16. What is the role of a Concrete Command?
17. Why is the Receiver required?
18. What is the role of the Invoker?
19. How does Command pattern support polymorphism?
20. How does MVC improve maintainability?
21. Give a real-world example of MVC.
22. Give a real-world example of the Command pattern.
23. Why can Command objects be more flexible than direct method calls?
24. Can multiple commands use the same Receiver?
25. How can MVC and Command patterns be used together?